

Data Breach Analysis — Guided Lab

Assess a leaked database, read-only, with Python

Day 3 · guided · from a raw SQL dump to an exposure report

You import a leaked SQL dump into a local database, discover its schema on your own, and quantify the exposure with Python (SQLAlchemy + pandas) — **read-only**, anonymising outputs. The instructor holds a methodology correction (WriteUp_FR_DataBreach.md).

Original brief: Case_Brief_original.pdf. This document adds step-by-step guidance, checks and a scoring grid. Work only with the dump provided for the exercise, in a **defensive** posture: assess impact, never re-identify or contact victims, never crack passwords.

Learning objectives

- Stand up a local MariaDB/MySQL and import a raw `.sql` dump.
- Connect Python to the DB with SQLAlchemy and load results into pandas.
- Explore an **unknown** schema (tables, columns, relationships) independently.
- Write analytical SQL (filter, dates, aggregates, joins) to produce metrics and a synthesis.
- Reason about GDPR impact and password-storage robustness — without any attack.

Setup

```
cd "Day_3/Data Breach Lab"
pip install sqlalchemy pymysql pandas
# a local DB, e.g. via Docker (see the Docker & n8n Lab):
docker run -d --name breachdb -e MARIADB_ROOT_PASSWORD=lab -p 3306:3306 mariadb:11
```

Step 0 — Import & sanity check (20 min)

0.1 Obtain the dump (`.sql`), create a database, and import it. Verify the import succeeded — a real dump has **hundreds** of tables. Write a query that returns the total table count.

```
docker exec -i breachdb mariadb -uroot -plab -e "CREATE DATABASE leak;"
docker exec -i breachdb mariadb -uroot -plab leak < dump.sql      # import
```

0.2 Before Python, peek at the dump header **without importing** (it tells you the CMS):

```
head -20 dump.sql          # -- MariaDB dump ... Database: ... -> platform fingerprint
grep -c '^CREATE TABLE' dump.sql
```

What CMS / platform does the header + table count suggest? Why does that matter for what you will find?

Step 1 — Connect & discover the schema (30 min)

Constraint: all analysis goes through Python (SQLAlchemy + pandas). Keep credentials in an environment variable, never hard-coded.

```
import os, pandas as pd
from sqlalchemy import create_engine, inspect
engine = create_engine(os.getenv("DATABASE_URL",
"mysql+pymysql://root:lab@127.0.0.1:3306/leak"))
insp = inspect(engine)
print(len(insp.get_table_names()), "tables")
pd.read_sql("SELECT COUNT(*) total FROM information_schema.tables WHERE
table_schema=DATABASE()", engine)
```

1.1 List every table with its estimated row count. Identify the **core** tables (users, profiles, messages) — how did you spot them (names, row counts, `LIKE '%user%'`)?

1.2 For 3–4 key tables, print the column structure, data types and a sample of rows. Document each. Watch for dates stored as **Unix timestamps**.

Step 2 — User accounts (25 min)

2.1 Total accounts, how many active, how many with a verified email (find the status columns).

2.2 Monthly registrations. Convert timestamps: SQL `FROM_UNIXTIME(...)` or pandas `pd.to_datetime(...)`. Plot or tabulate the trend.

2.3 Top 10 email domains. What do they tell you about the user base?

Step 3 — Profiles & relational joins (30 min)

3.1 Join user accounts to public member profiles (foreign keys). Validate on sample records — distinguish account id from profile id.

3.2 Demographics: breakdown by gender.

3.3 Ages from birth dates, grouped `<18 / 18–25 / 26–40 / 41–60 / 60+`. Handle missing/invalid values properly.

3.4 Top 10 most-viewed public profiles.

Step 4 — Messaging (20 min)

4.1 Total conversations and total messages. 4.2 Monthly message volume. 4.3 Top 10 most active conversations (by volume), with titles. 4.4 Top 10 most active users in messaging.

Step 5 — Dashboard, export & synthesis (25 min)

5.1 An executive-summary DataFrame: total accounts, active, verified, profiles, conversations, messages, earliest/latest registration.

5.2 Export at least one aggregated dataset (e.g. monthly registrations) to CSV that opens cleanly in a spreadsheet.

5.3 A 5–10 line synthesis: peak periods & trends; typical demographic; **GDPR/privacy vulnerabilities** found (sensitive fields, timestamp anomalies, ghost accounts, password storage).

Do not include raw personal data or unmasked credentials in your submission. Anonymise emails and names.

Deliverables

One zip `BREACH_<team>.zip`:

1. Modular, commented Python scripts (`.py`), split by step, credentials from env vars.
2. Exported `.csv` file(s).
3. A technical report (Markdown/PDF): methodology, schema-discovery process, queries, numeric results, analytical observations — **anonymised**.
4. `requirements.txt` (`sqlalchemy`, `pymysql`, `pandas`).

Grading grid (100 pts)

Item	Pts	What earns the points
Import & schema discovery	20	table count, core tables found, schema documented
User analysis	20	volumetrics, monthly trend, top domains
Profiles & joins	20	correct join, demographics, age brackets, missing-value handling
Messaging	15	volumetrics, top conversations/users
Dashboard & synthesis	20	KPI summary, CSV export, GDPR reasoning
Rigour	5	anonymised output, env-var credentials, read-only

Optional Extensions (Bonus)

- **Full-text search benchmark:** LIKE '%kw%' vs MATCH ... AGAINST on profile descriptions.
- **Data-quality audit:** aberrant birth dates, dormant unverified accounts, duplicate emails.
- **Password-storage assessment (read-only):** hash algorithm signatures, length, salt presence — comment on robustness **without any cracking**.
- **Emit STIX:** model the breached Identity + an Observed Data of the exposure as STIX (see the STIX Lab).